A BEGINNER'S FIELD GUIDE

Prompt engineering is *drafting*, not talking.

A language model has no idea what you want until you draw it. This guide walks through how to think about a prompt before you write one, the structural patterns that consistently work, and the difference between a vague request and a working blueprint — grounded in Anthropic's own guidance for prompting Claude.

Before worrying about syntax, get the right picture in your head of what a prompt actually is. Three models cover almost every situation you'll run into.

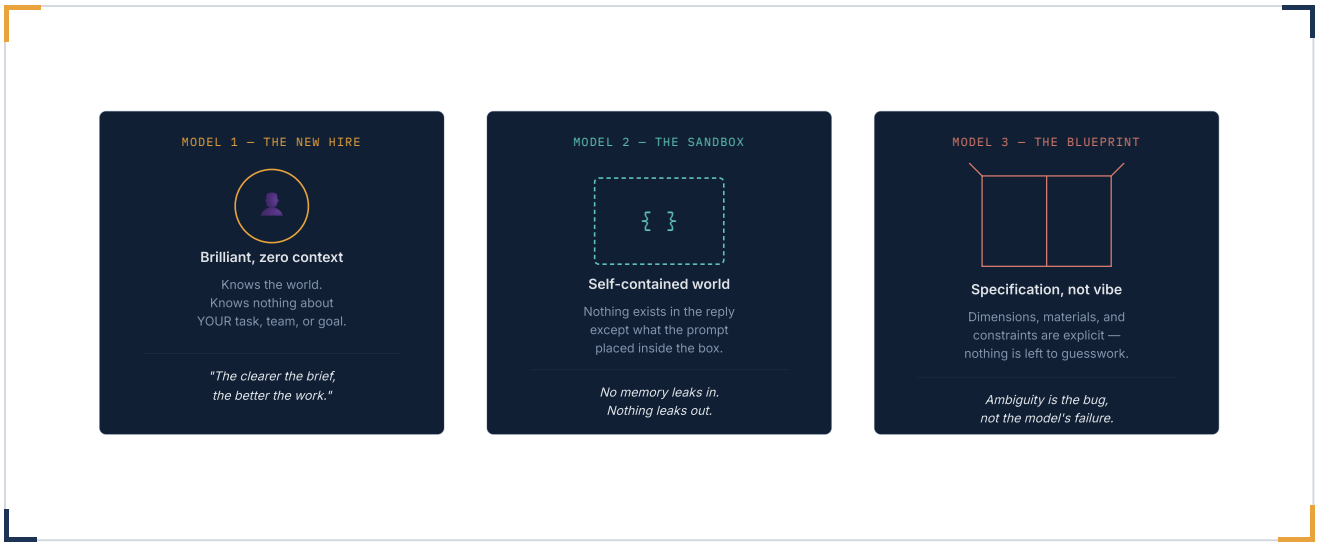

FIG. 01 — Three lenses for approaching any prompt-writing task

USE WHEN

You need judgment or a point of view

Hand the model a role with relevant expertise — "you are a senior tax accountant reviewing this filing" — so it weighs evidence and prioritizes the way that expert would, not as a generic assistant.

USE WHEN

You need one task, cleanly isolated

Give the model everything it needs inside the prompt and explicitly forbid outside assumptions. Good for classification, extraction, transformation — tasks with one right shape of answer.

USE WHEN

The output has to be exact

Spell out structure, format, and edge cases before you ask for the content. Treat every unstated detail as something the model is free to invent — because it will.

02 How To Actually Build One

Anthropic's own guidance frames this less as "find the magic words" and more as an engineering loop: define what success looks like, draft, test, and iterate. Here's the thinking sequence, step by step.

1

Define success before you write a word

What does a great answer look like, concretely? What does a bad one look like? If you can't tell the two apart on paper, the model won't be able to either. Write 2–3 example outputs you'd be happy with.

2

Pick the mental model that fits the task

Judgment call → persona. Isolated transformation → sandbox. Exact output shape → blueprint. Most real prompts blend two of the three.

3

Draft with structure, not prose paragraphs

Separate task, context, constraints, and examples into distinct labeled sections (headers or XML-style tags both work). A model parses structure far more reliably than it parses a wall of run-on instructions.

4

Add examples where words run out

If the desired format, tone, or edge-case handling is hard to describe in a sentence, show it instead. One well-chosen example is often worth three more paragraphs of instruction.

5

Test against real inputs, then patch the gaps

Run it on messy, boundary-case input — not just the clean example you used to write the prompt. Where it breaks, that's exactly where the prompt is ambiguous. Tighten there.

---

03 Six Core Prompting Patterns

These aren't competing techniques — they're tools that get combined inside a single prompt. A production prompt commonly uses four or five of these at once.

---

01 / ZERO-SHOT

`no examples given`

You describe the task directly and trust the model's general knowledge to fill the gaps — "summarize this email in two sentences." Fast to write, and the right default for simple, well-understood tasks. It struggles when the desired format or tone is unusual, because the model has nothing to anchor to except your wording.

---

02 / FEW-SHOT (ICL)

`in-context learning`

You show 2–5 input → output pairs before the real input, and the model latches onto the pattern rather than guessing at your intent from a description. This is the single highest-leverage technique for getting consistent formatting, tone, or classification behavior — because you're demonstrating, not describing.

03 / PERSONA / ROLE

`"you are a ___"`

Assigning a role — "you are a meticulous copy editor," "you are a skeptical code reviewer" — shifts what the model treats as relevant and how it weighs trade-offs. It's not roleplay for its own sake; it's a compact way to encode a whole set of priorities and standards without spelling each one out individually.

04 / CHAIN OF THOUGHT

`"think step by step"`

Asking the model to reason through intermediate steps before giving a final answer measurably improves accuracy on math, logic, and multi-step analysis. The model effectively gets to "show its work," which both improves the answer and gives you a trail to audit when something goes wrong.

05 / PROMPT CHAINING

`one task, several calls`

Break a complex job into smaller subtasks, each its own prompt, and feed one's output into the next — draft, then critique, then revise. Each step has a single focused objective, so nothing gets dropped in a long, overloaded instruction list. Slower and more expensive than one big prompt, but far more reliable.

06 / STRUCTURED TAGGING

`XML / markdown sections`

Wrapping distinct parts of a prompt — instructions, reference data, examples — in clearly labeled sections (e.g. `<instructions>`, `<data>`, `<examples>`) stops the model from mixing up what's an instruction and what's content to act on. Claude in particular was trained to track this kind of structure closely.

04 Chain of Thought — Making Reasoning Visible

The core idea: a model that writes out intermediate reasoning before committing to an answer is less likely to skip a step or jump to a plausible-sounding but wrong conclusion.

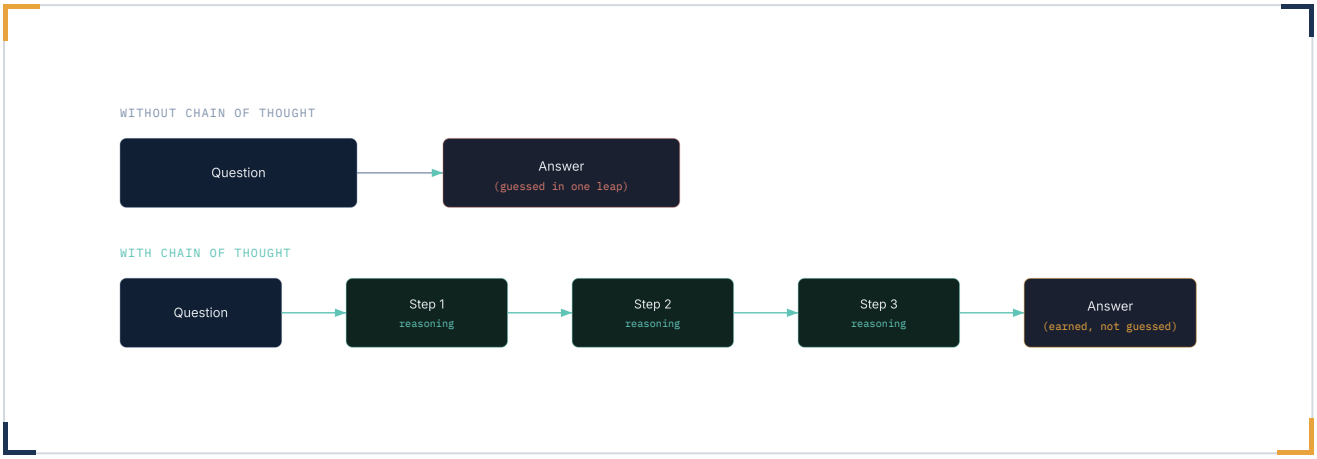

FIG. 02 — Direct leap to an answer vs. an explicit reasoning trail

WHEN IT HELPS MOST

Multi-step or ambiguous problems

Math, logic puzzles, multi-criteria decisions, debugging — anywhere the right answer depends on getting several intermediate steps right in order.

WHEN TO SKIP IT

Simple, single-step lookups

For trivial factual or formatting tasks, forcing reasoning steps just adds length and latency with no accuracy gain. Match the technique to task difficulty.

> *"Think step by step" is the simplest version. A stronger version names the steps explicitly: "First identify the variables, then the relationship between them, then compute, then state the answer."*

05 ReAct — Reasoning Interleaved With Action

Chain of Thought reasons about a static question. ReAct reasons about a changing world — it's the pattern behind tool-using agents: think, act, observe what happened, then think again.

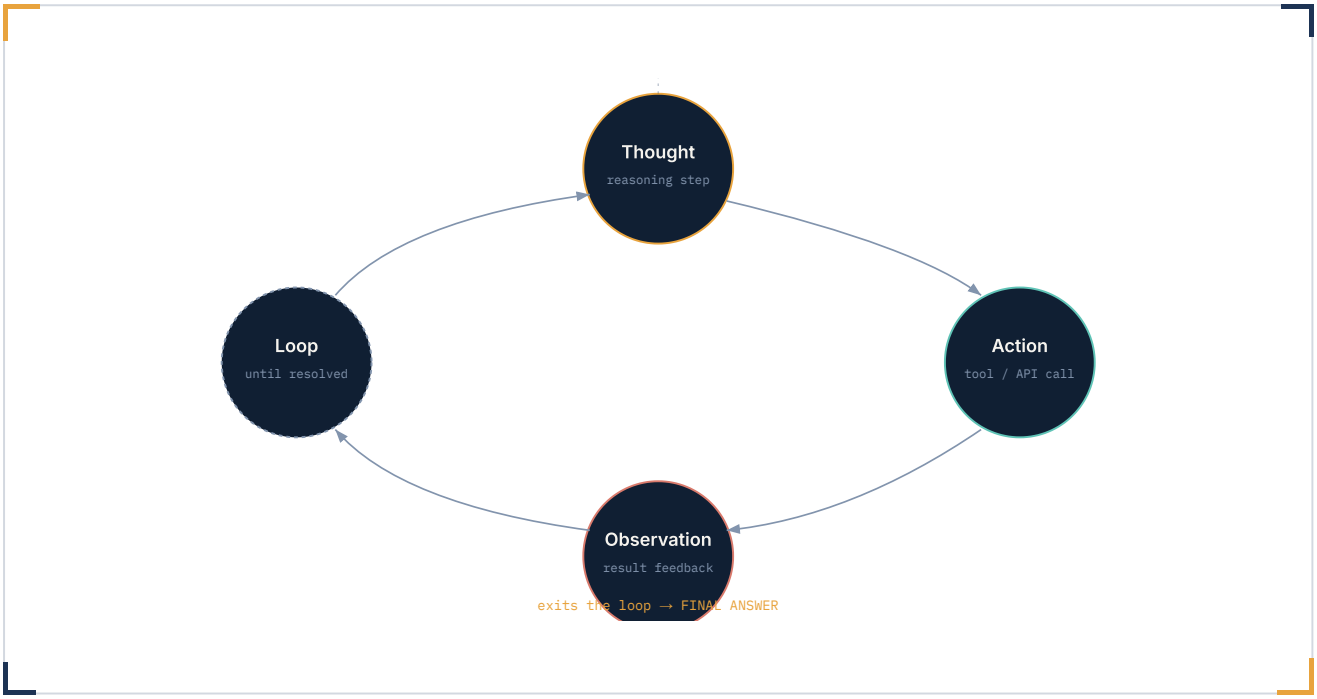

FIG. 03 — The ReAct cycle: think, act on the world, observe, repeat until done

THOUGHT

What do I need, and why

The model states its reasoning about the current state of the problem and decides what to try next — out loud, not silently.

ACTION

Call a tool or take a step

A real operation against the outside world: a search, a calculation, an API call, a file read — anything that changes what's known.

OBSERVATION

What came back

The result of that action is fed back in, and the loop repeats — reasoning again in light of new information — until a final answer is reached.

This is the backbone of every modern tool-using agent, including how Claude itself operates when it has access to tools: it doesn't write one long plan up front and execute blindly — it reasons, takes one action, looks at what happened, and re-plans.

---

06 Context Is the Real Material

Anthropic frames this as the natural evolution of prompt engineering: the prompt is only one slice of what's actually in the model's attention at any moment. Context engineering is about curating *everything* that lands in that window — not just the instructions.

LAYER	WHAT LIVES HERE	COMMON MISTAKE
System Prompt	Stable role, tone, and standing rules that apply to every turn	Cramming task-specific detail in here instead of the user turn
Instructions	What to do, right now, for this specific request	Vague verbs ("handle this") instead of a concrete action

Reference Data	Documents, retrieved facts, prior outputs the model should ground itself in	Dumping raw, unstructured walls of text with no labeling
Examples	Input → output pairs demonstrating the exact pattern wanted	Examples that contradict the written instructions
Tool Results	Output from actions already taken in this conversation	Keeping every raw result in context long after it's been used

> *The goal is the smallest set of high-signal tokens that maximizes the chance of the outcome you want — not the longest, most exhaustive prompt you can write.*

TOO RIGID

Hardcoded if-else instructions

Brittle prompts that try to enumerate every possible case break the moment reality presents a case you didn't think of.

TOO LOOSE

Vague high-level guidance

"Be helpful and thorough" gives the model no concrete signal about what a good output looks like for this specific task.

07 Worked Examples — Patterns In Code

Two of the patterns above, written as runnable prompts against the Claude API: a few-shot classifier, and a structured Chain-of-Thought + ReAct-style trace.

Few-Shot Classification

Three input → output pairs teach the exact JSON shape, then the model completes the pattern on a new input — no schema description needed beyond the examples themselves.

few_shot_classifier.py

```
import os
from anthropic import Anthropic

client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

system_prompt = (
    "You are a log classification parser. "
    "Output only the JSON object — no preamble, no explanation."
)

few_shot_prompt = """Input: Firewall blocked 450 repeated requests from 192.168.1
Output: {"event": "DOS_BLOCK", "severity": "HIGH", "source": "FIREWALL"}

Input: Broker node disk utilization crossed 89% threshold.
Output: {"event": "DISK_SURGE", "severity": "MEDIUM", "source": "OS_BROKER"}

Input: Gateway logged a routine SSL handshake renegotiation.
Output:"""

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=200,
    system=system_prompt,
    messages=[{"role": "user", "content": few_shot_prompt}],
)

print(response.content[0].text)
# → {"event": "SSL_RENEGOTIATION", "severity": "LOW", "source": "GATEWAY"}
```

Chain of Thought, Made Explicit

Naming the reasoning steps in the instructions — rather than just saying "think step by step" — gives a more auditable, consistent trace.

chain_of_thought.py

```
cot_prompt = """Solve this problem. Use exactly this structure:

Thought: state what quantity you need and which numbers are relevant
Action: do the calculation
Observation: state the resulting number plainly
Final Answer: the answer in the units the question asked for

Problem: A sensor array sends 80,000 packets per second, each
512 bytes. What is the required throughput in MB/s?"""

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,
    messages=[{"role": "user", "content": cot_prompt}],
)

print(response.content[0].text)
```

---

08 Before You Hit Send

A last pass before shipping a prompt — most failures trace back to one of these.

STRUCTURE

- [x] **Task is separated** from background context and from examples — not blended into one paragraph.
- [x] **Output format** is stated explicitly, not implied.

- [x] **Edge cases** you can predict are addressed up front, not patched after a failure.

JUDGMENT

- [x] **Right pattern chosen** — persona for judgment calls, examples for exact formatting, reasoning steps for multi-step logic.
- [x] **Tested on messy input**, not just the clean example used while writing it.
- [x] **Nothing critical** is left for the model to assume or guess.
